

Martín Alcaraz

Javier Reverte

Segunda práctica entregable - Sistema de recomendación para servicio de streaming musical

Explicación de los requisitos del sistema de recomendación

- **R1: Solo debe existir una única instancia del recomendador por usuario.**

En la clase **Recomendador** se ha utilizado el patrón de diseño **Singleton**, que garantiza que solo exista una instancia del recomendador durante toda la ejecución del programa.

Para ello se utiliza una variable de clase `_instancia` junto con el método de clase `obtenerRecomendador`, que controla la creación del objeto. Si la instancia aún no existe se crea, y si ya existe simplemente se devuelve la misma.

De esta forma se asegura que el sistema utilice siempre el mismo objeto recomendador y que la sesión de escucha del usuario se mantenga coherente durante toda la ejecución.

-
- **R2: Cada nueva canción escuchada debe actualizar los estadísticos de la sesión de escucha.**

La sesión de escucha del usuario se almacena en el atributo `sesion` de la clase **Recomendador**, que guarda las canciones escuchadas durante la ejecución del sistema.

Para calcular los estadísticos de la sesión se ha utilizado el patrón **Chain of Responsibility**, mediante las clases **EstadisticoSonoro** y **EstadisticoSentimental**.

Cada uno de estos componentes calcula de forma secuencial los estadísticos correspondientes a los atributos sonoros y sentimentales de las canciones escuchadas, como la media y la desviación típica.

Una vez terminado su cálculo, cada componente pasa el control al siguiente elemento de la cadena, permitiendo que el procesamiento de los datos se realice de forma ordenada y modular.

-
- **R3: El sistema debe permitir recomendar canciones, artistas o playlists.**

Para extender la funcionalidad del sistema se ha utilizado el patrón de diseño **Decorator**.

La clase base `RecomendacionBase` genera inicialmente una recomendación de canción. Posteriormente, mediante los decoradores **DecoratorArtista** y **DecoratorPlaylist**, se añaden recomendaciones adicionales de artistas y playlists sin modificar la lógica principal del recomendador.

Cada decorador envuelve al objeto de recomendación anterior y añade nueva información al resultado final. Este enfoque permite ampliar fácilmente las funcionalidades del sistema manteniendo una arquitectura flexible y modular.

· **R4: El sistema debe permitir tres estrategias de búsqueda en el catálogo.**

Para implementar las distintas formas de búsqueda en el catálogo se ha utilizado el **patrón de diseño Strategy**, que permite definir distintos algoritmos de búsqueda y seleccionar uno de ellos dinámicamente en función de las necesidades del sistema. Este patrón resulta especialmente adecuado en este caso, ya que el sistema debe poder cambiar la forma de seleccionar recomendaciones sin modificar la lógica principal del recomendador.

En primer lugar, se define una clase abstracta denominada **EstrategiaBusqueda**, que actúa como interfaz común para todas las estrategias. Esta clase establece el método abstracto **buscar**, que debe ser implementado por todas las estrategias concretas. De esta forma se garantiza que todas las estrategias proporcionen el mismo tipo de comportamiento, permitiendo que el sistema pueda utilizarlas de manera intercambiable.

A partir de esta clase base se implementan tres estrategias diferentes:

1. Búsqueda alfabética (BusquedaAlfabetica)

Esta estrategia ordena los elementos del catálogo (canciones, artistas o playlists) según su **título o nombre en orden alfabético**, utilizando la función `sorted` de Python. Una vez ordenados, se selecciona el primer elemento de la lista resultante. Este tipo de búsqueda permite recorrer los elementos del catálogo desde la letra **A hasta la Z**, simulando un criterio de búsqueda basado en orden alfabético.

2. Búsqueda temporal (BusquedaTemporal)

En esta estrategia los elementos del catálogo se ordenan según su **fecha de creación** (en el caso de canciones y playlists) o **fecha de nacimiento** (en el caso de artistas). Para ello se utiliza la función `getattr`, que permite acceder dinámicamente al atributo correspondiente. El orden se establece desde el elemento **más reciente al más antiguo**, de forma que el sistema prioriza contenidos más actuales dentro de las recomendaciones.

3. Búsqueda aleatoria (BusquedaAleatoria)

Esta estrategia selecciona un elemento del catálogo de forma **completamente aleatoria**, utilizando la función `random.choice`. Este método permite introducir variabilidad en las recomendaciones y evitar que el sistema siempre devuelva resultados similares, favoreciendo la exploración de nuevos contenidos.

Gracias al uso del patrón **Strategy**, el sistema puede cambiar el algoritmo de búsqueda simplemente instanciando una estrategia distinta (por ejemplo, `BusquedaTemporal` o `BusquedaAleatoria`) sin necesidad de modificar el resto de las clases del sistema. Esto aporta una mayor **flexibilidad, modularidad y facilidad de mantenimiento**, ya que permite añadir nuevas estrategias de búsqueda en el futuro sin alterar la estructura existente del recomendador.

· **Uso de funciones lambda y funciones de orden superior**

En varias partes del sistema se utilizan **funciones lambda y funciones de orden superior** para simplificar operaciones sobre listas de canciones.

Por ejemplo, para calcular la media de los atributos sonoros de una sesión se emplean expresiones como:

```
media = sum(map(lambda c: c.atributos_sonoros[k], canciones)) / len(canciones)
```

Este tipo de funciones permite realizar cálculos de forma más compacta y funcional, evitando la necesidad de bucles explícitos.

· **Uso de concurrencia con subrutinas asíncronas**

Para simular el comportamiento en tiempo real del sistema se utilizan funciones asíncronas mediante la librería **asyncio**.

En la función **simular_escucha_real** se utilizan pausas asíncronas (`await asyncio.sleep`) para simular el tiempo que transcurre entre la reproducción de canciones. Esto permite que el sistema procese la información de forma no bloqueante y simule el comportamiento de una aplicación real de streaming.

· **Gestión de errores y tests**

El sistema también incluye manejo de excepciones para evitar fallos durante la ejecución del programa. Por ejemplo, se controla que:

- la canción solicitada exista en el catálogo
- la sesión no esté vacía
- los atributos necesarios estén disponibles

Además, se han implementado **tests unitarios utilizando pytest**, definiendo al menos un test para cada clase principal del sistema para verificar su funcionamiento.

Mejoras y correcciones realizadas durante el desarrollo

Durante el desarrollo del sistema se realizaron varias mejoras sobre versiones iniciales del código con el objetivo de garantizar el correcto funcionamiento del sistema de recomendación y cumplir adecuadamente los requisitos del enunciado.

1. Corrección del cálculo de atributos medios de artistas y playlists

Inicialmente, las clases **Artista** y **Playlist** almacenaban únicamente una lista de canciones, pero no disponían de información agregada sobre sus características sonoras o sentimentales. Esto dificultaba la comparación entre los gustos del usuario y los artistas o playlists del catálogo.

Para solucionar este problema se añadió la propiedad **atributos_medios** en ambas clases. Esta propiedad calcula la media de los atributos de todas las canciones asociadas al artista o a la playlist.

De esta forma, el sistema puede representar el estilo musical de un artista o una playlist a partir de las características promedio de sus canciones, lo que permite comparar estos valores con los atributos medios de la sesión de escucha del usuario.

Este enfoque mejora significativamente la calidad de las recomendaciones, ya que el sistema puede determinar qué artistas o playlists presentan características musicales similares a las preferencias del usuario.

2. Corrección de la estrategia temporal en el patrón Strategy

Durante el desarrollo se detectó un problema en la estrategia de búsqueda temporal. En una versión inicial del código, el sistema intentaba ordenar todos los elementos del catálogo utilizando el atributo **fecha_creacion**.

Sin embargo, los objetos **Artista** no poseen este atributo, sino que cuentan con **fecha_nacimiento**, lo que provocaba un error cuando el sistema intentaba aplicar la estrategia temporal a una lista de artistas.

Para resolver este problema se modificó la implementación de la estrategia temporal utilizando la función **getattr**, que permite acceder dinámicamente al atributo correspondiente dependiendo del tipo de objeto.

De este modo, el sistema utiliza:

- **fecha_creacion** para canciones y playlists
- **fecha_nacimiento** para artistas

Esta solución permite que la estrategia temporal funcione correctamente para todos los tipos de elementos del catálogo.

3. Mejora del sistema de recomendación en los decoradores

En una versión inicial del sistema, los decoradores encargados de recomendar artistas y playlists seleccionaban elementos del catálogo sin verificar si realmente eran similares a las preferencias del usuario.

Para mejorar la calidad de las recomendaciones se introdujo un mecanismo de filtrado basado en la similitud entre los atributos medios de la sesión del usuario y los atributos medios de los artistas o playlists.

Este filtrado se implementa mediante la función de orden superior **filter**, utilizando una función **lambda** que compara las medias de los atributos.

De esta forma, el sistema selecciona únicamente aquellos artistas o playlists cuya media de atributos sea suficientemente cercana a la media de la sesión de escucha del usuario.

Esta mejora garantiza que las recomendaciones generadas tengan una mayor coherencia con los gustos del usuario.

4. Implementación de asincronía para simular procesamiento en tiempo real

Otra mejora importante fue la incorporación de **programación asíncrona** utilizando la librería **asyncio**.

En versiones iniciales del sistema, las canciones escuchadas por el usuario se procesaban de forma instantánea y secuencial. Para simular un entorno más realista de streaming musical, se implementó una función asíncrona que reproduce canciones con pausas temporales entre ellas.

Mediante el uso de **async def**, **await** y **asyncio.sleep**, el sistema simula la reproducción de canciones en tiempo real sin bloquear la ejecución.

Esto permite representar de forma más realista el comportamiento de una plataforma de streaming, donde las canciones se reproducen de forma progresiva y el sistema actualiza continuamente las recomendaciones en función de las nuevas escuchas del usuario.

A continuación, unas capturas de pantalla de los comandos en git utilizados para la creación del repositorio git y las sucesivas acciones para subir y editar los trabajos.

Creación del repositorio git

Empezamos nuestro trabajo creando el repositorio en git y asociándolo a un repositorio local remoto. Para ello ejecuto los siguientes comandos:

Primero, iniciamos git en mi directorio local:

```
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git init
Initialized empty Git repository in C:/Users/RosaB/OneDrive/Escritorio/2año/2cuatri/programacion_CD/practicas/entregables/entregable_2/.git/
```

Creamos el archivo que alojará el código fuente(manualmente): entregable1.py

A continuación, creamos el entorno virtual en mi repositorio:

```
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> python -m venv .
```

Ahora, después de crear el repositorio git, vamos a subir mi repositorio local a dicho repositorio remoto:

[Javierreverte-edu-upct-es/pcd_entregable2_2526_MartinAlcaraz_JavierReverte](https://github.com/Javierreverte-edu-upct-es/pcd_entregable2_2526_MartinAlcaraz_JavierReverte)

```
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git remote add origin https://github.com/Javierreverte-edu-upct-es/pcd_entregable2_2526_MartinAlcaraz_JavierReverte
```

```
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git branch -M main
```

```
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git remote set-url origin https://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@github.com/Javierreverte-edu-upct-es/pcd_entregable_1_Javier
```

Por último, creamos una rama "development" para realizar los diferentes commits:

```
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git checkout -b development
Switched to a new branch 'development'
```

Y la subimos al repositorio remoto:

```
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git push -u origin development
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'development' on GitHub by visiting:
remote:   https://github.com/Javierreverte-edu-upct-es/pcd_entregable2_2526_MartinAlcaraz_JavierReverte/pull/new/development
remote:
remote: To https://github.com/Javierreverte-edu-upct-es/pcd_entregable2_2526_MartinAlcaraz_JavierReverte
* [new branch]      development -> development
branch 'development' set up to track 'origin/development'.
```

Una vez que tenemos el repositorio creado, comenzamos a realizar el trabajo.

Durante la realización del trabajo hemos creado dos tags (v1.0 y v2.0). El primero ha sido al termina por primera vez el código que posteriormente ha sufrido diferentes modificaciones. El segundo, después de corregir ciertos fallos y dar por terminado el código final. Posteriormente hacemos el pytest). La fijación de los tags ha sido de la siguiente forma:

```
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git tag v1.0
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git push origin v1.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Javierreverte-edu-upct-es/pcd_entregable2_2526_MartinAlcaraz_JavierReverte
* [new tag]          v1.0 -> v1.0
```

```
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git tag v2.0
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git push origin v2.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Javierreverte-edu-upct-es/pcd_entregable2_2526_MartinAlcaraz_JavierReverte
* [new tag]          v2.0 -> v2.0
```

Una vez terminado el código, procedemos al uso de pytests para hacer unos tests de prueba y darle la forma final al archivo. Hemos instalado pytest porque lo requería de la siguiente forma:

La ejecución de pytest mostró que todos los tests se ejecutan correctamente sin errores.

```
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> pip install pytest
Requirement already satisfied: pytest in c:\users\rosab\anaconda3\lib\site-packages (7.4.4)
Requirement already satisfied: iniconfig in c:\users\rosab\anaconda3\lib\site-packages (from pytest) (1.1.1)
Requirement already satisfied: packaging in c:\users\rosab\appdata\roaming\python\python312\site-packages (from pytest) (24.1)
Requirement already satisfied: pluggy<2.0,>=0.12 in c:\users\rosab\anaconda3\lib\site-packages (from pytest) (1.0.0)
Requirement already satisfied: colorama in c:\users\rosab\appdata\roaming\python\python312\site-packages (from pytest) (0.4.6)
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> pytest codigo_test.py
===== test session starts =====
platform win32 -- Python 3.12.3, pytest-7.4.4, pluggy-1.0.0
rootdir: C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2
plugins: anyio-4.2.0
collected 21 items

codigo_test.py ..... [100%]

===== 21 passed in 0.32s =====
```

A continuación, hacemos el commit:

```
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git add .
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git commit
-m "pytest funciona"
[development d9a0991] pytest funciona
3 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 __pycache__/codigo_TAG4.cpython-312.pyc
create mode 100644 __pycache__/codigo_test.cpython-312-pytest-7.4.4.pyc
create mode 100644 __pycache__/codigo_test1.cpython-312-pytest-7.4.4.pyc
(base) PS C:\Users\RosaB\OneDrive\Escritorio\2año\2cuatri\programacion_CD\practicas\entregables\entregable_2> git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 20 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 16.93 KiB | 2.12 MiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/Javierreverte-edu-upct-es/pcd_entregable2_2526_MartinAlcaraz_JavierReverte
 4c32f15..d9a0991  development -> development
```